

MLOps Lab Project — File Usage & Logic Guide

A file-by-file explanation of the uploaded AWS MLOps Customer Churn project, including how the files connect, what each file does, and the MLOps concepts demonstrated.

Project goal

Build, evaluate, register, package, deploy, and monitor a customer-churn machine-learning model using Python, MLflow, DVC, FastAPI, Docker, AWS, Terraform, and GitHub Actions.

1. End-to-End Architecture

Data generation → data validation/preprocessing → model training → MLflow tracking → model evaluation/quality gate → model registry → FastAPI → Docker → ECR → EC2 → monitoring/drift detection.

Conceptual flow:

data/generate_data.py → data/churn.csv → src/data_prep.py → src/train.py → artifacts + MLflow → src/evaluate.py → registry → api/main.py → Docker → AWS ECR/EC2 → monitoring/drift_check.py

2. data/generate_data.py

Purpose: Creates a synthetic customer-churn dataset so the lab can run without an external production data source.

Logic: generate(rows=5000, seed=42) creates customer features such as tenure_months, monthly_charges, total_charges, support_tickets, num_products, contract_score, usage_score, and satisfaction_score. The target column is churn (0/1). make_classification() creates an intentionally imbalanced dataset; weights=[0.73, 0.27] means roughly 73% non-churn and 27% churn. Some values are transformed/clipped to resemble business data.

MLOps concept: data ingestion simulation / reproducible dataset generation.

3. data/churn.csv

Purpose: The generated dataset consumed by the ML pipeline.

Logic: Each row represents a customer. Feature columns are the model inputs and churn is the supervised-learning target. This file is the hand-off point between data generation and model training.

MLOps concept: dataset artifact; in a real system this could be replaced by data stored in S3/data warehouse.

4. src/data_prep.py

Purpose: Centralizes loading, validation, train/test splitting, and feature scaling.

Logic: load_raw() reads the CSV and verifies the target exists. validate() checks for nulls, invalid tenure values, valid churn labels, and a reasonable churn rate. split_and_scale() separates X and y, performs an 80/20 train-test split using stratify=y, and applies StandardScaler. Keeping preprocessing in one module helps training and serving remain consistent.

MLOps concept: data-quality gates, reproducible preprocessing, leakage prevention.

5. src/train.py

Purpose: Trains the Random Forest model and records experiments with MLflow.

Logic: The flow is: load data → validate → split/scale → build RandomForestClassifier → train → predict → calculate accuracy/precision/recall/F1/ROC-AUC → log parameters and metrics to MLflow → save model/scaler/metadata artifacts → optionally register the model. class_weight='balanced' helps with the imbalanced churn target.

MLOps concept: experiment tracking, model training, artifact management.

6. artifacts/

Purpose: Stores outputs required to evaluate and serve the trained model.

Logic: model.joblib = trained model. scaler.joblib = exact preprocessing scaler used during training. feature_names.json = expected feature order. metrics.json = evaluation metrics. The scaler and feature order are critical because production inference must apply the same transformations used during training.

MLOps concept: model artifact management and reproducibility.

7. src/tune.py

Purpose: Performs hyperparameter optimization using Optuna.

Logic: Instead of manually trying many Random Forest configurations, Optuna evaluates trials with different values such as n_estimators, max_depth, and min_samples_leaf. The objective is to find a configuration that improves the selected metric, then retrain/register the best model.

MLOps concept: automated hyperparameter optimization.

8. src/evaluate.py

Purpose: Acts as the model quality gate before promotion.

Logic: Reads artifacts/metrics.json and checks thresholds such as $F1 \geq 0.70$ and $ROC-AUC \geq 0.75$. If the model fails, the process exits with a non-zero status so CI/CD stops. This prevents a technically deployable but poor model from automatically reaching production.

MLOps concept: automated model validation and promotion control.

9. tests/test_data.py

Purpose: Tests dataset quality and preprocessing assumptions.

Logic: Checks conditions such as expected row count, presence of the churn column, no null values, valid target values, realistic churn distribution, and expected train/test behavior. These are data-quality tests that can run in CI.

MLOps concept: data testing.

10. tests/test_model.py

Purpose: Tests model quality and reproducibility.

Logic: Compares the trained model against a simple majority-class baseline and checks that the model provides sufficient improvement. It also checks deterministic behavior when seeds/configuration are fixed.

MLOps concept: model testing and reproducibility.

11. api/schemas.py

Purpose: Defines and validates the FastAPI request/response contract.

Logic: Pydantic schemas describe the eight customer input features and their acceptable ranges/types. Invalid API requests can be rejected before reaching the model.

MLOps concept: reliable model-serving interface.

12. api/main.py

Purpose: Serves the trained model through FastAPI.

Logic: GET /health provides a health check. POST /predict accepts customer features, orders them correctly, applies the saved scaler, runs the model, calculates churn probability, and converts it into a prediction using a threshold such as 0.5. The application can load a production model from MLflow and use local artifacts as a fallback for the lab.

MLOps concept: online inference / model serving.

13. monitoring/drift_check.py

Purpose: Detects changes between training data and newer production-like data.

Logic: For each feature it compares baseline statistics such as mean/std with current statistics. A z-shift style calculation is used; a large shift (for example >2) can be reported as potential drift. The script provides a simple classroom implementation that could later publish alerts/metrics to CloudWatch or another monitoring system.

MLOps concept: data drift monitoring.

14. monitoring/baseline.json

Purpose: Stores baseline statistics from the training dataset.

Logic: Contains reference statistics such as feature means and standard deviations. New data is compared against these values to identify distribution changes.

MLOps concept: production monitoring baseline.

15. orchestration/pipeline.py

Purpose: Connects individual scripts into an end-to-end training workflow.

Logic: Conceptually: generate data → optionally version/push data with DVC → train or tune → run quality gate → register the accepted model. It converts separate scripts into a repeatable pipeline.

MLOps concept: pipeline automation.

16. orchestration/training_flow_prefect.py

Purpose: Provides an alternative workflow using Prefect.

Logic: Tasks can represent data generation, training, and quality gates. Prefect adds orchestration capabilities such as task retries, scheduling, execution tracking, and workflow visibility.

MLOps concept: workflow orchestration.

17. .dvc/config.template

Purpose: Introduces DVC configuration for dataset versioning.

Logic: Git versions source code while DVC versions large data artifacts. An S3 bucket can act as the DVC remote. This makes it possible to associate a model/training run with a specific dataset version.

MLOps concept: dataset versioning.

18. Dockerfile

Purpose: Packages the API and model runtime into a reproducible container image.

Logic: Installs the Python dependencies, copies the application/model artifacts, and starts Uvicorn on port 8000. The container becomes the deployable unit.

MLOps concept: reproducible deployment artifact.

19. docker-compose.yml

Purpose: Runs the API and MLflow locally as separate services.

Logic: The API can point to MLflow using an internal Docker network address such as `http://mlflow:5000`. This gives the lab a simple local architecture with MLflow tracking/model registry plus the inference API.

MLOps concept: local multi-service environment.

20. infra/main.tf

Purpose: Defines AWS infrastructure using Terraform.

Logic: Creates the core AWS resources used by the lab: S3 for data/artifacts, ECR for the Docker image, IAM permissions/role, EC2 for compute, and security-group/network rules. The EC2 instance can run Docker, MLflow, and the FastAPI service.

MLOps concept: Infrastructure as Code and cloud deployment.

21. infra/variables.tf

Purpose: Defines configurable Terraform inputs.

Logic: Values such as AWS region, project/environment name, EC2 instance type, key pair, and allowed IP CIDR can be supplied as variables rather than hardcoded throughout the infrastructure.

MLOps concept: reusable IaC configuration.

22. infra/outputs.tf

Purpose: Prints useful deployment values after terraform apply.

Logic: Typical outputs include the EC2 public IP, MLflow URL, API URL, S3 bucket name, and ECR repository URL. These outputs make the deployment easier to consume from the command line.

MLOps concept: deployment discoverability.

23. .github/workflows/ci-cd.yml

Purpose: Automates testing, training, quality gates, image creation, and deployment.

Logic: Typical flow: checkout → install dependencies → lint/test → generate/train → evaluate quality gate → build Docker image → authenticate to ECR → push image → connect to EC2 → pull and restart the application. Deployment is normally restricted to the main branch.

MLOps concept: CI/CD and continuous model delivery.

24. Makefile

Purpose: Provides short commands for common lab operations.

Logic: Commands such as make data, make test, make train, make tune, make gate, make pipeline, make api, make docker-build, and make docker-run wrap longer Python/Docker commands. It improves usability for students and developers.

MLOps concept: developer automation.

25. requirements.txt

Purpose: Pins the Python libraries required by the project.

Logic: Key libraries include pandas/numpy for data processing, scikit-learn for ML, MLflow for tracking/registry, Optuna for tuning, FastAPI/Uvicorn/Pydantic for serving, pytest for testing, boto3 for AWS integration, and DVC for data versioning.

MLOps concept: reproducible runtime dependencies.

26. README.md

Purpose: Acts as the project's entry-point documentation.

Logic: Documents project purpose, directory structure, setup, commands, deployment, architecture, and cleanup. A good README lets another learner reproduce the lab without relying on the original developer.

MLOps concept: operational documentation.

27. How the Files Connect

Phase 1 — Data

generate_data.py → churn.csv → data_prep.py (load, validate, split, scale)

Phase 2 — ML

data_prep.py → train.py → Random Forest → metrics + artifacts + MLflow

Phase 3 — Quality

metrics.json → evaluate.py → PASS/FAIL

Phase 4 — Registry

accepted model → MLflow Model Registry → production version

Phase 5 — Serving

FastAPI schemas → main.py → scaler → model → prediction

Phase 6 — Container/AWS

Dockerfile → Docker image → ECR → EC2 → FastAPI/MLflow

Phase 7 — CI/CD

GitHub → GitHub Actions → tests → training → quality gate → Docker → ECR → EC2

Phase 8 — Monitoring

production/current data → drift_check.py → compare with baseline.json → detect drift → potential alert/retraining

28. MLOps Concepts Demonstrated

Area	Project implementation
Data versioning	DVC + S3
Data validation	data_prep.py + tests/test_data.py
Experiment tracking	MLflow
Model training	RandomForestClassifier
Hyperparameter tuning	Optuna / tune.py
Model artifacts	joblib + JSON
Model quality gate	evaluate.py
Model registry	MLflow Model Registry
Model serving	FastAPI
Containerization	Docker
Container registry	AWS ECR
Compute	AWS EC2
Object storage	AWS S3
Permissions	AWS IAM role
Infrastructure as Code	Terraform
CI/CD	GitHub Actions

Area	Project implementation
Monitoring	Drift detection + baseline statistics
Workflow orchestration	Pipeline script + Prefect

29. Recommended Learning Order

1. generate_data.py
2. data_prep.py
3. train.py
4. artifacts and MLflow
5. evaluate.py
6. tune.py
7. tests/
8. api/schemas.py + api/main.py
9. Dockerfile + docker-compose.yml
10. DVC
11. Terraform/IAM/S3/ECR/EC2
12. GitHub Actions CI/CD
13. monitoring/drift_check.py
14. Prefect orchestration

30. Production-Grade Improvements to Discuss

The project is excellent for a lab, but several areas should be strengthened before production: place application compute behind an ALB/private networking where appropriate; restrict MLflow/API exposure; add authentication/authorization; avoid embedding model artifacts in the image when registry-based loading is the intended production path; use centralized logs/metrics; add stronger drift and model-performance monitoring; use a managed or highly available MLflow backend; and use secure CI/CD credentials such as OIDC rather than long-lived AWS keys.

31. One-Minute Interview Explanation

“This project implements an end-to-end MLOps pipeline for customer churn. We generate and validate the dataset, version data with DVC, preprocess and train a Random Forest model, track experiments and artifacts in MLflow, tune hyperparameters with Optuna, and enforce F1/AUC quality gates before model promotion. The approved model is exposed through FastAPI, packaged with Docker, stored in ECR, and deployed to EC2 using Terraform and GitHub Actions. The production side also includes a baseline-based drift check so changes in incoming feature distributions can trigger monitoring and potentially retraining.”